

Chetna

AI-powered predictive flood monitoring and early-warning system — complete framework

Problem statement IS-12 | Sponsor: Ernst & Young (EY) | Theme: AI & Intelligent Systems (Smart Cities)

Scope: one pilot city | 7 days | team of 4

One-line summary: Rain forecast + terrain + sensors → AI risk map → official approves → citizen gets an SMS/WhatsApp alert with a safe route.

1. Problem and objective

Problem: Cities respond to floods after they happen. Warnings are city-wide and generic, so people in the danger zone do not know their own risk or the safe way out.

Objective: Predict which neighbourhoods will flood 1 to 6 hours ahead, show it to authorities, and send targeted warnings with safe routes to citizens.

Success looks like:

- Flags real flood-prone spots earlier than a simple rain-threshold rule.
- Fewer false alarms.
- Alert to safe route in under a minute in the demo.

2. Layers of the system

Layer	What it does
1. Data	Collects rain forecast, terrain, roads, sensors
2. Processing	Builds the city grid and the static vulnerability map
3. Prediction	ML model gives risk per cell for +1h, +3h, +6h
4. Correction	Sensor readings adjust the risk in real time
5. Decision	Routing, at-risk assets, human approval
6. Communication	SMS/WhatsApp alerts (optional web push)
7. Interface	Authority dashboard and citizen view

3. Modules: inputs, work, outputs

Module	Input	Work	Output
M1 Data ingestion	Open-Meteo, DEM, OSM, WorldCover	Download and clean; freeze CRS and grid	Files/tables ready to use
M2 Static risk	DEM, land use	Elevation, slope, flow accumulation, imperviousness per ~200 m cell	Vulnerability score per cell
M3 Label generator	Past rain, static risk, 5-10 known waterlogging spots	Simple runoff proxy (rain x imperviousness – drain capacity, routed by flow accumulation)	Training labels
M4 Risk model	Static score, forecast 1/3/6h, past 24h rain, sensor value	XGBoost classifier	Low, Moderate, High or Severe per cell per horizon

5. Tech stack

Layer	Tool	Why
Language	Python 3.10+	One language for the whole team
Data fetching	requests, openmeteo-requests	Rain forecast from Open-Meteo (no API key)
GIS and terrain	GeoPandas, Rasterio, pysheds or richdem	Read DEM, slope, flow accumulation, build grid
Roads and shelters	OSMnx + NetworkX	OSM roads, hospitals, schools; A*/Dijkstra routing
ML model	XGBoost or scikit-learn Random Forest	Fast, works on small data
Explainability	SHAP (or feature importance)	Shows why a zone is flagged
Sensor simulator	Plain Python (paho-mqtt optional)	Realistic water levels from rainfall
Database	SQLite (or PostgreSQL + PostGIS)	Zero setup, enough for a demo
Dashboard	Streamlit + Folium	Map, time slider, Approve button, no frontend code
Alerts	Twilio (SMS + WhatsApp sandbox)	Quick setup, free trial
Backup channel	Telegram Bot API	Free fallback for the demo
Optional push	Firebase Cloud Messaging	Only if time is left on Day 6
Tools	GitHub, conda/venv, Jupyter/Colab, QGIS	Collaboration and checking GIS layers

Avoid for now: React, Kafka, Airflow, Docker/Kubernetes, LSTM/TFT. Mention them on a 'future scope' slide instead.

Data sources (all free)

Data	Where
Rain forecast	Open-Meteo API
Past rain for training	IMD gridded data (imdlb) or NASA IMERG
Elevation	Copernicus DEM (or SRTM)
Roads, hospitals, shelters	OpenStreetMap
Land use	ESA WorldCover
Past flood spots	News reports, Sentinel-1 flood maps
River gauges (optional)	India-WRIS / CWC

6. ML design

Features: elevation, slope, flow accumulation, imperviousness, forecast rain (1/3/6h), past 24h rain, nearest sensor reading.

Labels: from the M3 runoff proxy, calibrated against known waterlogging spots. State this openly.

Model: XGBoost with four classes (Random Forest as fallback).

Validation: hold out entire wards, not random cells, so the test is not on training locations.

Metrics: recall (POD), false-alarm rate, F1, lead time. Recall matters most because a missed flood costs the most.

Baselines to beat:

- Rain-threshold rule (alert if forecast is above X mm).
- Static low-lying map only.

7. Dashboard pages

Page	Content
Live map	Risk heatmap, time slider (+1h/+3h/+6h), sensor markers
At-risk list	Hospitals, schools, underpasses, low-lying colonies in High/Severe zones
Why flagged	Short explanation per zone
Alert centre	Draft message preview, Approve/Reject, alert log
Citizen view — Find my safest location	Enter a location, get risk level and the safest route out (or a WhatsApp/Telegram flow)
Demo control	'Simulate heavy rain' button

8. Alert design

Channels: SMS and WhatsApp via Twilio (Telegram as demo fallback). Web push through Firebase only if time is left.

Content: risk level, area, expected time, what to do, safe route link.

Rules: send by ward and severity; de-duplicate to avoid spamming; human approval before any mass alert.

Privacy: store phone numbers only for consented users, and never show them in the demo.

Why SMS stays essential: networks get congested in floods, and SMS often gets through when mobile data does not.

9. Repository structure

```
chetna/
|-- data/          raw and processed files, sample gauge data
|-- src/
|   |-- ingestion/  weather, dem, osm
|   |-- static_risk/ grid, flow accumulation
|   |-- model/      labels, train, predict, explain
|   |-- sensors/    simulator, correction
|   |-- routing/    graph, safe_route
|   |-- alerts/     templates, twilio_client, telegram_client
|   |-- db/         sqlite helpers
|-- app/           Streamlit dashboard
|-- notebooks/     experiments, backtest
|-- requirements.txt
`-- README.md
```

Database tables (SQLite)

Table	Columns
cells	id, geometry, elevation, slope, flow_acc, vulnerability
forecasts	timestamp, rain_1h, rain_3h, rain_6h
sensor_readings	sensor_id, timestamp, level, status
risk_predictions	cell_id, timestamp, horizon, level, probability
alerts	id, zone, level, message, status, approved_by, timestamp

10. Evaluation plan

- 1 Pick one past heavy-rain event in your pilot city.
- 2 Replay it hour by hour through the system.
- 3 Compare flagged cells against known waterlogging spots.
- 4 Fill one table: our system vs the rain-threshold baseline (recall, false alarms, lead time).
- 5 Add an ablation line: with sensor correction vs without.

Do not quote accuracy numbers before you have run this backtest. Claim a measured improvement over defined baselines, not 'better than every existing system'.

11. Team roles

Person	Owns
A: Data / GIS	M1, M2, grid, ward mapping
B: ML	M3, M4, M7, backtest
C: Backend	M5, M6, M8, M9, database
D: Frontend / pitch	M10, citizen view, slides, demo video

12. 7-day timeline

Day	Goal
1	Choose city, freeze grid/CRS, download all data, set up repo and Twilio sandbox
2	Static risk map, forecast fetch, sensor simulator, dashboard skeleton
3	Model trained, risk on map with time slider, sensor correction
4	Routing, alert flow with Approve button, explanations
5	Backtest and results table, full end-to-end run
6	Bug fixes, Hindi templates, 'Simulate storm' button, slides, optional river mode
7	Rehearse, record backup demo video, no new features

13. Demo flow (2-3 minutes)

- 1 Show the city map on a calm day, all green.
- 2 Click 'Simulate heavy rain'. Rain rises, sensors update, low-lying zones turn red with a 3-hour warning.
- 3 Dashboard lists the hospitals and schools at risk and explains why each zone is flagged.
- 4 Click Approve. A phone on screen receives the SMS/WhatsApp message.
- 5 Show the safe evacuation route avoiding flooded roads — the safest location it leads to.
- 6 Show the results table against the rain-threshold baseline.

14. Risks and fixes

Risk	Fix
GIS libraries will not install	Use conda; test on Day 1
Twilio WhatsApp sandbox issues	Use SMS or Telegram as fallback
No real drain data	Use DEM and OSM proxies and say so openly
Live APIs fail during the demo	Cache sample responses; keep a backup video
Weak labels	Calibrate against known hotspots; show uncertainty
Scope creep	Freeze features at end of Day 6

15. Optional add-on: river flash-flood mode

Only if time allows (Day 5-6). An upstream river gauge acts as a trigger. The system estimates travel time to each downstream ward as distance along the river divided by flood-wave speed, then sends the same SMS/WhatsApp alert with a route away from the river. It reuses the alert and routing code.

Reference event: the 26 August 2026 Nepal-Tibet flash flood (Bhote Koshi / Trishuli), caused by a glacier collapse near the border. A rain-based model cannot predict such an event. But once an upstream trigger fires, downstream villages can be warned, and the flood took time to travel down the valley. Real gauge data would come from Nepal's DHM or, for Indian rivers, CWC and India-WRIS. Detailed data for this event may not be public yet, so treat it as a what-if story, not your main test.

16. Limits to state in the pitch

- Sensors are simulated in the prototype (allowed by the brief).
- Drain capacity is estimated, not surveyed.
- Labels are proxy-based and validated against known hotspots.
- It predicts rain-driven flooding. It does not predict glacier or dam collapses, though an upstream-gauge mode can warn downstream areas.
- If phone networks fail, SMS alerts fail too; a real deployment would add sirens or loudspeakers.

17. How we differ from existing systems

Existing approach	What it misses	How Chetna fills the gap
Physics-based city models (e.g. iFLOWS-type systems)	Costly and slow to set up in a new city	Hybrid ML on open data; faster and cheaper to deploy
River flood forecasting (CWC, Google Flood Hub)	Mostly riverine, not street-level waterlogging	Ward-level urban waterlogging focus
Alert channels (e.g. Sachet)	Delivery only, no prediction	Prediction, routing and alerts in one loop
Map-only dashboards	No approval workflow or citizen routes	Approve button, SMS/WhatsApp, safe routes to the nearest safe location

Verify current details of each existing system before quoting them in your pitch.

18. Pitch structure (7-8 slides)

- 1 Problem, with a real flood example
- 2 Why existing solutions fall short
- 3 Our solution in one line
- 4 How it works (the flow diagram)
- 5 Live demo
- 6 Results table vs baseline
- 7 Impact and scalability
- 8 Limits and next steps

19. Future scope (one slide only)

Real IoT sensors, municipal drain data, radar nowcasting, LSTM models, citizen photo reports, a what-if planner for drain desilting, and integration with government alert systems.